

Final Exam Topics

8. Object oriented modeling

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Object oriented modeling

The view systems of object oriented modeling: static model (class diagram, object diagram, package diagram, component diagram); dynamic model (state machine diagram, sequence diagram, communication diagram, use case diagram).

1 The view systems of object oriented modeling

Object oriented programming = data abstraction + abstract data type + type inheritance

- Abstraction

At a given level of programming, the neglecting of details irrelevant from the point of view of the solution.

- Data type

A data type is an ordered pair (A, F) , where A is the set of data and F is a finite set of operations. ($\forall f \in F : f : A^n \rightarrow A$) There are simple and composite data types.

Type class: A complex description of the type, which serves the abstract (PAR + EXP) and concrete (IMP + BODY) description of the given data type. So:

Type class = (PAR, EXP, IMP, BODY), where:

PAR = <properties of the parameters >

EXP = <the set of type objects and the name, syntax, semantics of their operations >

IMP = <services taken over from another class >

BODY = <representation, realization of the type class >

- Type inheritance

The two main forms of type inheritance: *specialization* and *reuse*

1. The subclass takes over the abstract properties and uses them in the export part
2. The type set, the parameter sets, the names of the operations can be redefined.
3. type set of the subclass = type set of the superclass

The consequences of specialization:

- polymorphism

Every variable has two types: *static* (received during declaration) and *dynamic* (at the moment of declaration it coincides with the static one, but can later change, if we assign a subclass instance to a superclass instance)

- dynamic binding

The assignment to the function/attribute of the computation rule corresponding to the dynamic type, at the moment of execution

View systems

- usage aspect

To whom does the system provide a service? (Persons or other systems, programs)

- Structural, static aspect

What units are there, what is their task, and how do they connect to each other?

- Dynamic aspect

How do the individual subunits behave, what states do they take on, under the effect of what events do they switch them? What is the mechanism of cooperation between the units? How do the messages play out between them in time?

- Implementation aspect

What software components are there, and what connections are there between them?

- Environmental aspect

What hardware and software resources does the system require during the solution?

2 Static model (class diagram, object diagram, package diagram, component diagram)

2.1 Class diagram

A connected graph describing the structure of the solution, to whose nodes we assign the classes, and to whose edges the relations between the classes (inheritance, association, aggregation, composition).

Only one class diagram belongs to the system.

Classes

A class looks like this:

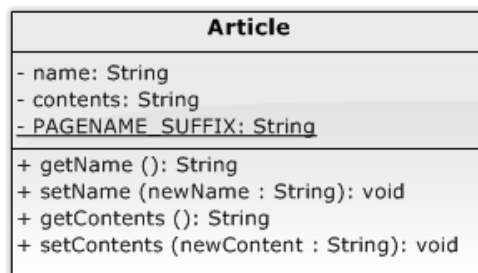


Figure 1: Class

The rectangle describing the class is divided into 3 parts.

- The name of the class goes into the first part.
If the class is abstract, we write its name in italics.
- The attributes of the class go into the second part.
The format of an attribute: *Attribute name : Type* Staticness is indicated by underlining. The visibility of the attributes can also be shown: *public (+), private (-), protected (#)*
- The methods of the class go into the third part.
The format of the methods: *Method name(Parameter list):Return value*, where the parameter list consists of parameters of the format *Parameter name:Type*. Abstractness, staticness, and visibility are indicated identically to what was described above.

Relationships between classes:

- inheritance
Denotes an abstraction relationship between two classes

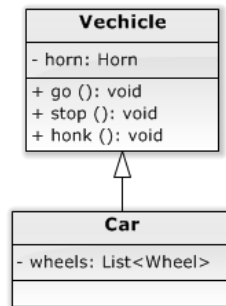


Figure 2: Inheritance

- association
This is the most general relation between two classes. The association is an abstract relation between two classes that expresses a bidirectional pairing. The abstract nature of the relation means that the concretization of the relation is realized in the connection of objects of the classes.

The association can have:

- Name, identifier
- Direction
- Multiplicity
Either a single value, or an interval. The * symbol gets a distinguished role; its meaning: any number, even zero. (E.g.: 3, 1..4, 5..*, *)
- Role
- Navigability
Of the paired classes only one knows the other. (If we do not show it, we assume mutual reachability)



Figure 3: Association

- aggregation

Aggregation is a special association that expresses a whole-part relationship. However, if an aggregation relation holds between two classes, the objects of the two classes can also exist independently of each other (this is called a so-called loose containment relation). The relation is also characterized by transitivity, and common attributes or services. Different aggregates can have common components.

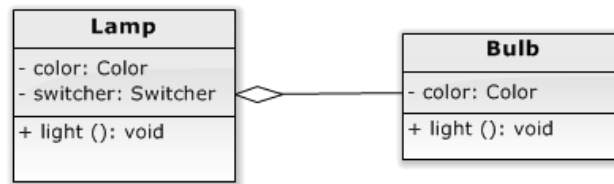


Figure 4: Aggregation

- composition

Composition is a special aggregation that denotes *physical* containment. It is no longer characterized by the independent existence of the objects; the two objects are created and destroyed at the same time. So the containing object has to take care of creating and destroying the contained one. A component can belong to at most one containing object. The composition relationship and the attribute-type relationship between two objects are semantically identical.

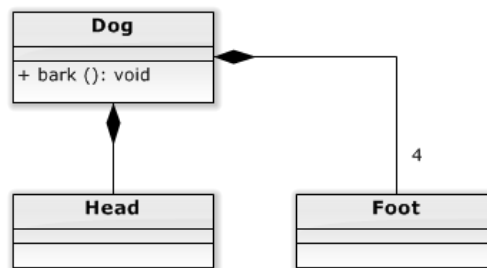


Figure 5: Composition

2.2 Object diagram

The object diagram is a simply connected graph to whose nodes we assign the objects, and to whose edges the connections between the objects.

Different object diagrams can belong to the system at different moments in time. (However, each of them has to conform to the class diagram)

Objects

Objects, like classes, are described by a rectangle. Such a rectangle has two parts. In the first part is the name of the object (optional) and its type, in the following format: *Object name : Type*, which we adorn with underlining. The attributes of the object and their values can go into the second part, in the following format: *Attribute name=value*.

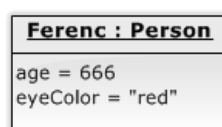


Figure 6: Object

Relationships between objects

The relations connecting the objects are the same as those on the class diagram (inheritance has no meaning at this level):

- association

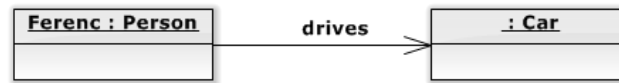


Figure 7: Association

- aggregation

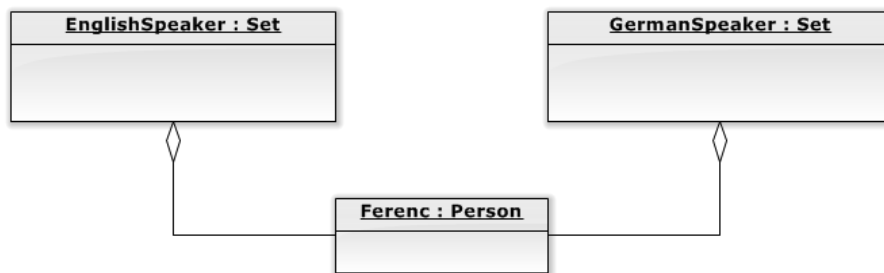


Figure 8: Aggregation

- composition

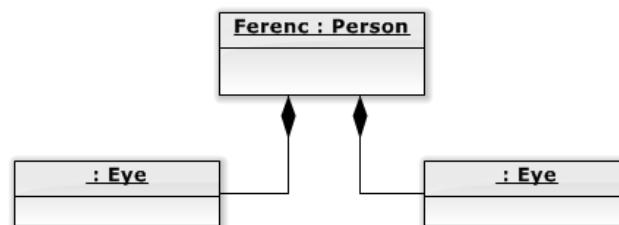


Figure 9: Composition

2.3 Package diagram

The package diagram is basically used for grouping other model elements, and as such it is also used in every development phase. During the production of the use case model it was already mentioned in connection with the use case inventory; in this chapter we summarize the detailed rules. The symbol of the package in UML is a “tabbed” rectangle. The label of the tab can be the name of the package, but it can also be written into the rectangle. Above the package name we can give a stereotype, below it a constraint. The content of the package can be any model element given textually or with a diagram, or a set of them. The previously mentioned use case inventory, for example, consists of packages containing use cases. Packages can also contain further packages. A model element can belong to only one package. Thus the packages form a tree-structured hierarchy, similarly to the elements of file systems, or to Java packages.

The package marked with the «system» stereotype is the top-level package representing the whole application. The dependencies between packages can be represented with a dashed, open-arrowhead arrow. The nature of the connection can be made precise with a stereotype. The UML package also represents a namespace, so within a package there cannot be two model elements with the same name. If we want to refer to a model element in

another package, we can do so by its qualified name. The qualified name contains the names of the packages through which we can reach the given model element. The package names are separated from each other by ::.

To the elements of the package we can also assign public(+) and private (-) visibility, similarly to the elements of classes. Referring by qualified name can be cumbersome because of its length, and every such reference can be affected by the reorganization of the packages. Therefore, instead, it is simpler and more practical to mark the references between packages with the arrow (drawn with a dashed line) denoting dependency and with stereotypes (importing). The imported element becomes visible to the importing package. The usable stereotypes:

«import»: public import, the importing package can further export it

«access»: private import, only the importing package can see it.

Its notation:

- A dashed arrow pointing from the importing package directly to the imported element
- An arrow from the importing package to the package containing the imported element, with the name of the element to be imported written after the stereotype
- An arrow drawn between the two packages, without an element name, means the import of all elements of the entire package.

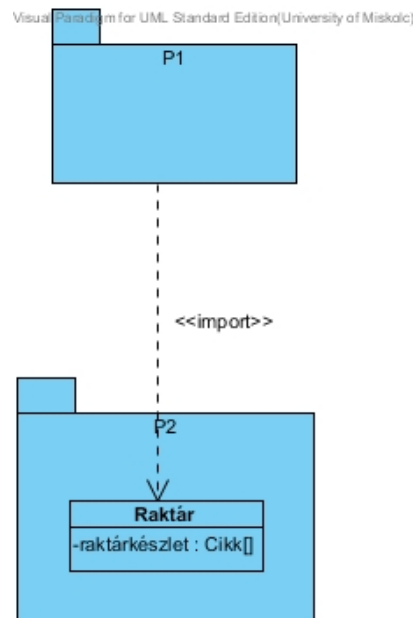


Figure 10: Package diagram: importing an element

2.4 Component diagram

The component in UML is a universal unit that represents some set of services and encloses them in a unit. Other components can access the services through interfaces. A component can be replaced with another if it has the same interface and provides the same services.

A component can contain other components.

The concept of components is well usable in every phase of development. Thus it can mean a component in the implementation sense (for example an Enterprise JavaBean), but also a complete subsystem (for example accounting).

The symbol of the component is a rectangle marked with the «component» stereotype, or with an icon. The component establishes connection with the outside world through ports. To every port one or more interfaces can be connected. The interface providing services to the outside world (provided interface) is denoted with a circle, and the one required by the component from others (required interface) with a half-circle. The connections

between components can be represented by the joining of the interfaces. A provided interface can be connected to only one required interface. (The notations also illustrate this well.)

The notation of the component diagram is illustrated by the figure below, which is a detail of the component diagram of the case study.

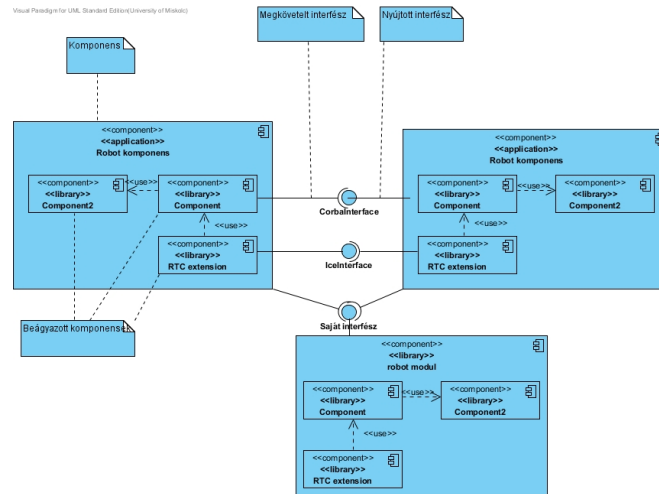


Figure 11: Component diagram

3 Dynamic model (state diagram, sequence diagram, collaboration diagram, activity diagram)

3.1 State diagram

The state diagram is a connected directed graph to whose nodes we assign the states, and to whose edges the events. (Between two vertices we can mark several state transitions, since one can be created under the effect of several events)

State

We characterize the state of the object with the n-tuple of the concrete values of the attributes.

The state has an identifier, which is most often the name of the state (but it can be the invariant itself, or the concrete value of the attributes). An event creates and destroys a state. The state persists as long as the attributes satisfy the invariant describing the state. The state is denoted with a rounded rectangle, in which we show the identifier.

Special (outside-the-system) states: Initial state, Final state

Event

We call an event the activity, occurrence, that changes the state of some object.

The event can be parameterized or parameterless, and it can have a precondition. There is an ordering among the events, so an event can have a preceding event. We can describe an event as follows:

<event>(<parameters>)[<condition>]/<preceding event>

We write the events onto the state transitions between the states.

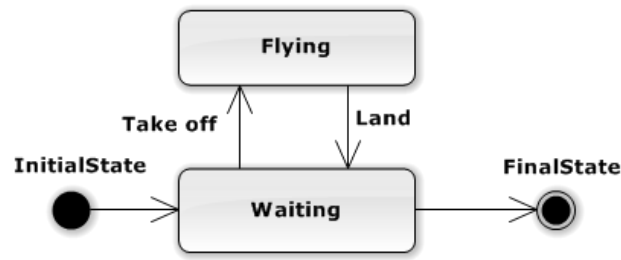


Figure 12: State machine of an airplane

3.2 Sequence diagram

The sequence diagram illustrates the temporal course of the message exchanges between objects.

Class role

The role of the class is embodied by one or more objects that behave conformly from the point of view of message sending.

Class role lifeline

The lifeline means the existence of the class role in time.

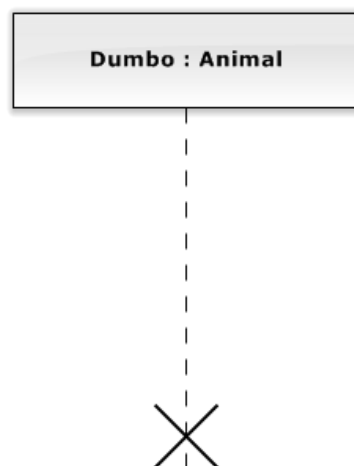


Figure 13: Lifeline

Activation lifeline

The activation lifeline means the state when the embodiments of the class role perform operations, or are under the control of other objects.

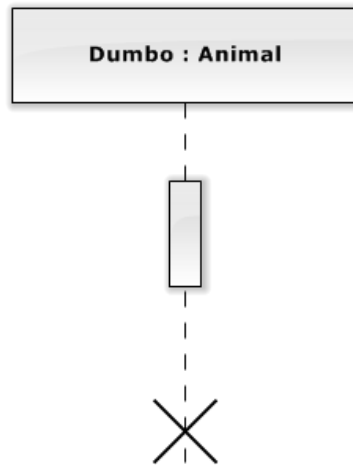


Figure 14: Activation lifeline

Message

The message is the form of information transfer between objects. The purpose of sending the message is for the object to set the other object into operation. The message can be created between those objects that are connected in the object diagram. The message has an identifier (name, text), and can have a parameter, sequence number.

3.3 Collaboration diagram

The collaboration diagram is meant to show how the objects of the classes cooperate, through the exchange of what messages this cooperation is realized. (Only those objects are relevant whose classes are connected by an association relationship in the class diagram. The diagram shows this connection and the message exchanges belonging to it, so the collaboration diagram can be regarded, in a certain sense, as an extension of the object diagram.)

The sending of a message is shown by an arrow that is placed next to the association and points toward the addressee. The identifier of the message is located along the arrow. The message can have an argument and a result. These we place next to an arrow starting from a small circle, where the arrow shows the direction of the flow of information.

3.4 Activity diagram

The activity diagram (activation diagram) illustrates the steps of solving the problem, together with the control flows running in parallel.

If an activity is followed directly by another activity, then we connect the two activities with an arrow. If an activity passes data (an object) to another activity, then a dashed arrow leads from the sending activity to the rectangle representing the object, and a dashed arrow points from the rectangle to the receiving activity. In the rectangle, between square brackets, we can also give the state, status of the object.

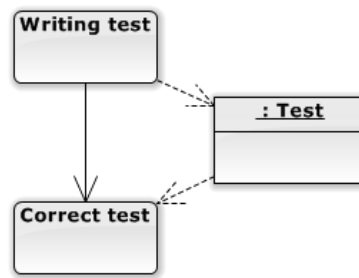


Figure 15: Object passing

It is possible to execute different activities upon the fulfillment of certain conditions, or to bind the execution of activities to conditions. Then we have to place a diamond in the diagram, onto whose outgoing arrows we write the conditions.

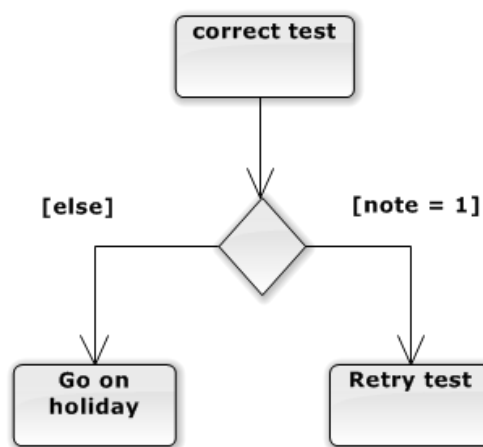


Figure 16: Representation of a condition

3.5 Communication diagram

The communication diagram, similarly to the sequence diagram, is able to represent message exchanges between objects; however, in this case we place the emphasis not on the temporality of the message exchanges, but on the connections between the objects participating in the message exchanges. On the sequence diagram the connection of the objects appears only indirectly, through which elements there is interaction between. On the communication diagram the cooperating objects are connected with the association-denoting lines (arrows) already familiar from the class diagrams. The temporal order of the messages is indicated by the numbering of the messages.

On the communication diagram an actor can also appear, as the source or addressee of a message.

It can be indicated if an object is active (capable of sending a message on its own), or if a message is directed not to one object but to a set of objects belonging to the same class (multiobject).

As an example, let us draw the communication diagram of a cash withdrawal from a bank's ATM. We assume that the customer has already identified themselves by inserting the card and entering the PIN code, and has chosen cash withdrawal from among the possible functions.

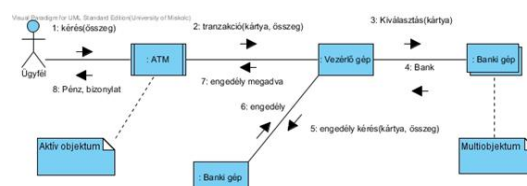


Figure 17: Communication diagram

4 Use case diagram

The use case diagram is meant to illustrate, from the point of view of the users, how the system works, independently of how it realizes its services.

The parts of the diagram:

- use cases
- users
- usage relations

The use cases are summaries of the functions of the system, service units. This unit is a sequence of actions by which the system cooperates with a group of users.

A use case is denoted with an oval shape. We enclose the use cases in a rectangle; this indicates the boundaries of the system.

The users are units outside the given system; they can be other program systems, subsystems, classes, or persons. These fill an actor role. On the diagram we denote them with a stick-figure.

The usage relations connect the use cases with the users. The relations can also be connected with each other, which can be shown in the diagram. The possible relations are the following:

- association
Indicates a connection between a user and a use case. (Simple line)
- generalization
One use case is the more general form of the other. (Simple line, with a white triangle at the end)
- extension
One use case extends the other. During this it inserts behaviors at specified insertion points. (Dashed arrow with `<<extend>>` label)
- inclusion
One use case contains the behavior of the other (Dashed arrow with `<<include>>` label)

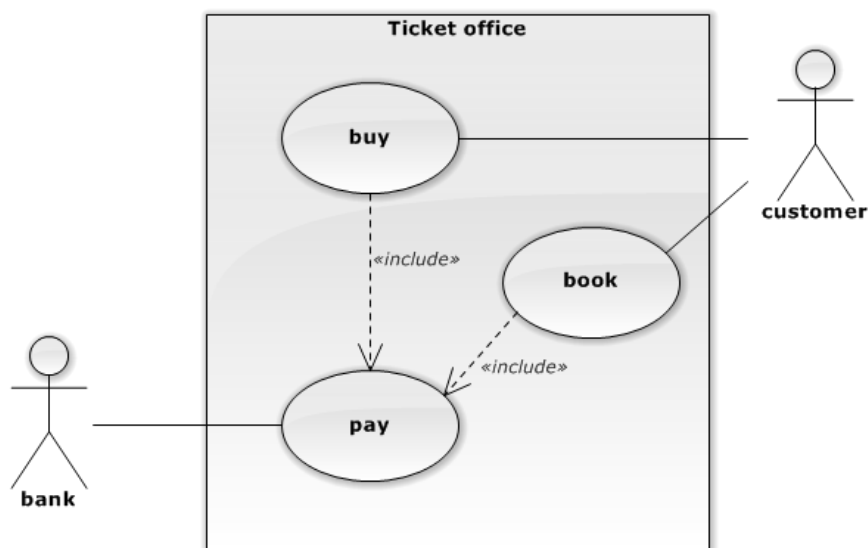


Figure 18: Use case diagram